Project Development Phase
GenAI Functional & Performance Testing

Date	[To be filled by team]
Team ID	[To be filled by team]
Project Name	Sustainable Waste Management Assistant Using Generative AI
Maximum Marks	[To be filled by evaluator]

GenAI Functional & Performance Testing:

Project team shall fill the following information in the GenAI functional and performance testing template. Test scenarios below are based on the actual implemented endpoints in app.py (Flask backend) and the Groq API integration (services/groq_service.py).

Test Case ID	Scenario (What to test)	Test Steps (How to test)	Expected Result	Actual Result	Pass/Fail
FT-01	Text input validation (waste item name)	Send POST /scan with a valid item name (e.g. "plastic bottle"); then send it with an empty/missing item field	Valid item is accepted and processed; empty/missing item returns HTTP 400 with error "Please enter a waste item."		
FT-02	Input length validation	Send POST /scan with an item name over 100 characters	Request is rejected with HTTP 400 and error "Waste item name is too long."		
FT-03	Missing request body handling	Send POST /scan with no JSON body at all	Request is rejected with HTTP 400 and error "Request body is missing."		
FT-04	AI content generation (disposal guidance)	Send POST /scan with a valid waste item and check the response	Response returns success: true along with the item and AI-generated analysis (disposal instructions, hazard warnings, recycling suggestions) from		

			generate_waste_guide()		
FT-05	Groq API failure handling	Simulate a Groq API/service failure (e.g. invalid API key) and call /scan	Endpoint catches the exception and returns HTTP 500 with a graceful error message instead of crashing		
FT-06	History, centers & dashboard data retrieval	Call GET /api/get-history, GET /api/get-centers, and GET /api/dashboard-data	Returns Firestore-backed data when available for the given user; otherwise falls back to demo data (get_demo_history / get_demo_centers / get_demo_dashboard) without erroring		
PT-01	Response time test	Measure the time taken for POST /scan to return a response for a typical item	Response should return within an acceptable time (team to define an acceptable threshold, e.g. under 3-5 seconds)		
PT-02	Concurrent request handling	Send multiple POST /scan requests to the Flask backend at the same time	API continues to respond correctly without crashing or significant slowdown		
PT-03	CORS / cross-origin access	Call the backend API from the React frontend running on a different port (localhost:5173 to localhost:5000)	Requests succeed without CORS errors, since flask_cors.CORS(app) is enabled		

Note: "Actual Result" and "Pass/Fail" columns are left blank for the project team to fill in after running these tests locally, since they require executing the live application and a valid Groq API key.